USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

The Interrupt Matrix, and Four Ways to Deliver an Edge to Nobody

UM-NATOS-023

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-023	1.0 · 2026-08-16	**Infrastructure verified; the first consumer is not**	Internal

1. Abstract

Until this work, **every peripheral in this kernel was polled**, and nothing said so. The Level-3 vector served exactly one source — CCOMPARE1, which is internal to the Xtensa core and reaches the CPU without passing through any routing at all. No peripheral interrupt had ever been routed, and the absence was invisible because the only peripherals were a display written on demand and a touch panel sampled at 100 Hz.

That is a hard wall for anything serviced on the hardware's schedule rather than ours — which is everything the sensor and audio work in §8 needs.

This report covers the matrix (`kernel/intr.c`), which **works and is verified**, and its first intended consumer, PENIRQ, which **does not and is switched off**. The second half is the more useful one: four distinct ways an interrupt was delivered correctly to something that could not act on it, each of which read back from every register as a complete success.

2. Sources are not lines

On the ESP32 a peripheral cannot reach the CPU by itself:

	range	fixed by	meaning
source	0–69	silicon	what raised it — GPIO, I2S0, UART1
line	0–31	silicon	what the CPU sees; its priority *level* and edge/level *type*

A DPORT map register per source holds the line number. These are different numbering spaces, and confusing them is the classic failure: writing a source number into `INTENABLE` enables an unrelated line while the peripheral stays silent, and every register involved reads back exactly as intended.

Line 23 is used here because it is level-triggered at level 3 — so it shares the existing Level-3 vector and its proven context save rather than needing a second one. A line's level is a property of the silicon, not a choice.

3. The vector could only ever have been right by accident

`_handler_level3` called `timer_isr` unconditionally. Correct while CCOMPARE1 was the only source that could reach level 3, and a **clock corruption** the moment a second one is routed: `timer_isr()` advances the

tick deadline by a whole interval every time it runs, so any GPIO interrupt would silently push the clock into the future.

That is the same defect as the `task_yield()` bug in UM-NATOS-014, arrived at from the opposite direction, and it would have been just as quiet — sleeps running long, timeouts running loose, and every frame rate measured against a clock that drifts.

The fix is one line of assembly: dispatch on the `INTERRUPT` register rather than on an assumption about who could have fired.

4. The defence against a hang

An enabled, pending, **unhandled** level-triggered line is not a glitch — it is a permanent hang. Nothing clears the condition, so returning re-enters the handler immediately and forever. There is no output from that state and no watchdog reach: the watchdog is fed by a task, and tasks have stopped running.

`intr_dispatch()` therefore **masks** such a line and counts it. The kernel loses that interrupt and keeps running, which is recoverable and inspectable rather than a silent brick.

5. Four ways to deliver an edge to nobody

The theme: at every stage, an edge was generated correctly, routed correctly, and consumed by something that could not act on it. **No register ever read back wrong.** Only counters distinguished these.

5.1 Delivered to a halted CPU

`GPIO_PINn_INT_ENA` is a 5-bit field at 17:13 selecting which CPU and which kind of interrupt receives the pin. Bit 13 was chosen as the low bit of the field, matching the vendor header's `PRO_CPU_INTR_ENA` being `BIT(0)`.

Real taps latched `GPIO_STATUS1` bit 4. The map register read 23. `INTENABLE` had bit 23. The CPU's `INTERRUPT` register never showed it.

The pin was being delivered to the **APP CPU, which this kernel never starts** — arriving perfectly at a core that was halted.

> **The measured answer is bit 15.** `intr_selftest()` settles it by injecting an edge through `GPIO_STATUS1_W1TS` once per candidate bit and reporting which the handler actually saw. Running `irqtest` re-derives the constant on any board rather than trusting the comment that records it.

Injecting the edge in software mattered for a second reason: it isolates the status→CPU path from the pad→status path. The alternative was asking someone to tap a panel five times while watching a serial log.

5.2 The driver interrupting itself

`touch.c` already knew that "a conversion in progress drives PENIRQ regardless of whether anything is touching". That is as true of the *interrupt* as of the *level*, and the driver had not been read that way.

Every `touch_read()` manufactured its own edges — **3,917 across a handful of taps, none caused by a finger.** Each one set the "edge arrived with nobody waiting" latch, so the idle wait returned immediately, forever. The driver had interrupted itself into a busy loop.

Fixed by masking the pin for the duration of the burst. An edge genuinely missed there costs nothing: the burst only runs while the task is awake and sampling.

5.3 Re-arming against a pin a finger is holding down

The mask was then released at the end of `touch_read()`, which looked right and was the same bug moved. The pad is still **LOW** at that point if a finger is on it, and arming falling-edge detection against an already-low pin latches an edge immediately.

Result: 100 edges, 0 wakes — every one self-inflicted, every one while the task was awake with no waiter registered.

The interrupt is only ever useful **while the task is idle and the pen is up**. Arming it anywhere else can only manufacture events, so arming now happens in `touch_irq_wait()` and nowhere else.

5.4 A diagnostic that was itself wrong

The first register dump read `0x3FF44078` for the PRO CPU's masked status and reported a confident zero. The per-CPU status registers sit between `STATUS1` and `PIN0` in an order taken from memory, and that address is most likely the *other* CPU's copy.

Printing the whole range and letting the value identify the register found bit 4 set at `0x3FF44074` — which is what pointed at §5.1.

> A diagnostic derived from the same faulty recollection as the code it is checking will agree with the code and both will be wrong. Dump the range, not the address you believe in.

6. What is verified

```
irqtest   -> bit 15 SERVICED          injected edge reaches the handler
intr      -> tick line 15 serviced N  dispatch does not disturb the clock
             spurious 0               nothing masked defensively
             armed rb 0x00008100      INT_TYPE=falling + enable, read back
                                      from inside the armed window
```

The last line matters as method: sampling `GPIO_PIN36` from the shell returned zero three times running against a window covering 93% of the cycle. That is not chance — the shell's sampling is correlated with the touch task being awake. Only the armed task could answer without the measurement being correlated with its own subject.

7. What is NOT verified, and why it is switched off

A finger has never once been observed to wake the touch task. With everything above confirmed, real taps produced 24 touch events and 30 low PENIRQ readings while the armed edge detector latched nothing. That gap is unexplained.

The touch task is therefore back to polling — byte for byte its previous behaviour — and `touch_irq_wait()` is intact, one line from reinstatement.

Shipping touch input on an interrupt whose end-to-end path has never been observed to work would trade a mechanism that demonstrably works for one that only should.

The most likely remaining suspects, untested:

- the pad→status edge path may need `FUN_IE` or a GPIO-matrix input selection that plain reads do not require — `gpio_read()` working does not prove the edge detector is fed
- the arm/disarm boundary may be clearing a finger's edge: `gpio_int_disable()` clears status, and it runs the instant the sleep expires
- GPIO 34–39 are input-only RTC pads and may route their edges differently from the ordinary bank

8. Why this was built first

The alternative was ADC — smaller, and with an onboard LDR to prove it. The matrix was chosen because it **cannot be bolted on later without revisiting every driver**, and because PENIRQ offered a real, low-rate, forgiving consumer already on the board.

That reasoning half survives. The infrastructure is real and the next peripheral inherits it. But the "forgiving consumer" turned out to be the hardest part, and an ADC would have proved the same matrix against something with no edge semantics to get wrong.

9. Metrics

Quantity	Value
Peripheral interrupts routed before this	0
CPU line used	23, level 3, level-triggered
GPIO source	22
PRO CPU enable bit	15 (**measured**, not read)
Self-inflicted edges before masking	3,917
Assembly changed	1 call
Distinct delivery failures found	4
Finger-driven wakes observed	**0**

10. What this does not establish

- **No peripheral interrupt has yet driven anything.** §7. The matrix is proven by injection only.
- `task_wake()` **has never woken a task.** It exists, it is correct by inspection, and no interrupt has ever called it successfully.
- **One line, one handler.** There is no sharing, no priority among sources, and no way to route two peripherals to one line.
- **Nothing is re-entrant.** Handlers run at level 3 with interrupts masked, so a slow one delays the tick, and nothing measures how long any of them take.

- **The APP CPU is still never started**, so half of every `INT_ENA` field is addressing a core that does not exist as far as this kernel is concerned.

11. References

- UM-NATOS-014 — the tick deadline defect §3 would have recreated
- UM-NATOS-017 §4.1 — PENIRQ, and why the level is trusted over the Z channels
- UM-NATOS-017 §7.4 — the calibration whose pair check is the same idea as §5.4
- `kernel/intr.c` — routing, dispatch, the hang defence, `irqtest`
- `kernel/gpio.h` — the interrupt registers, and the measured bit 15
- `kernel/touch.c` — the masking of §5.2/§5.3 and the disabled wait of §7